Gradient Descent

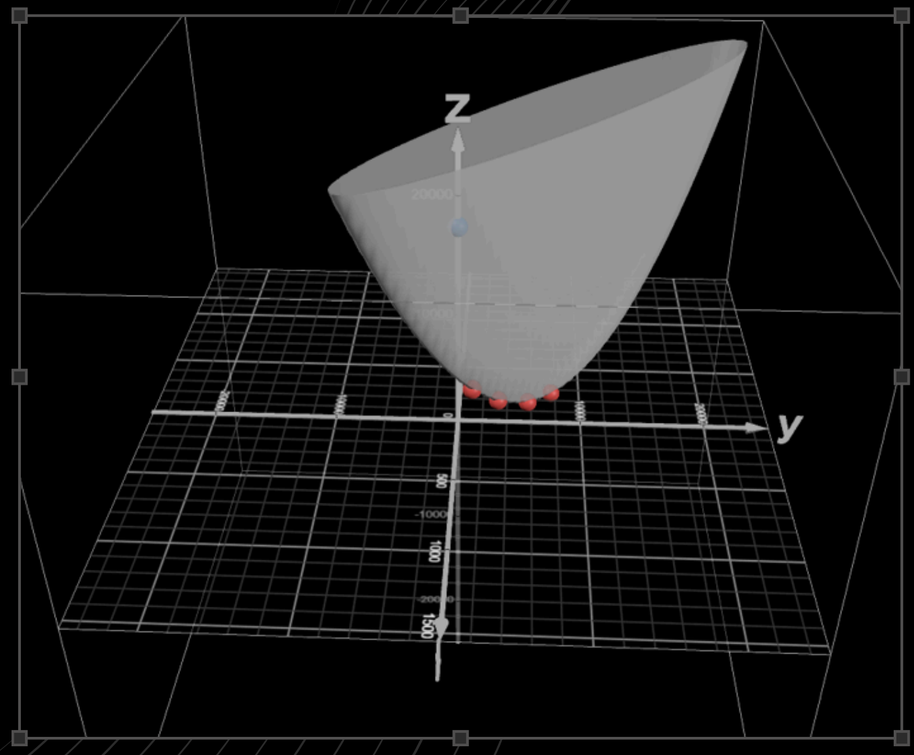


Table of content

Data Source

Dataset: metro_Interstate_traffic_volume (Westbound traffic in Minneapolis-St Paul, MN, from 2012-2018) link: https://archive.ics.uci.edu/dataset/492/metro+interstate+traffic+volume

it had 48205 data; I chose 10 data points chosen under similar weather conditions to isolate the relationship between time and traffic

Extracted Pairs (Time (x_i) vs. Traffic Volume (y_i)):

$$\begin{bmatrix} x_i & y_i \end{bmatrix} = \begin{bmatrix} 7 & 5972 \\ 9 & 5545 \\ 10 & 4378 \\ 11 & 4545 \\ 12 & 4902 \\ 14 & 4721 \\ 19 & 3783 \\ 21 & 2030 \\ 21 & 2179 \\ 22 & 1992 \end{bmatrix}$$

Prediction Model

Model Type: Simple Linear Regression

Equation: $$\text{Predicted Traffic Volume} = a \cdot (\text{Hour of Day}) + s$$

Initial Weights in Java: a = 1.0, s = 0.0

Error Function

Type: Sum of Squared Residuals

Formula:

$$F(a, s) = \sum_{i=1}^{10}((ax_i + s) - y_i)^2$$

This expression can be graphed as convex paraboloid guaranteeing a single unique global minimum where both partial derivates (respect to a and s) reach zero.

Derivative Calculations

Gradient with respect to intercept (s):

$$\text{sumGradS} = \sum_{i=1}^{10} 2((ax_i + s) - y_i)$$

Gradient with respect to slope (a)

$$\text{sumGradA} = \sum_{i=1}^{10} 2x_i((ax_i + s) - y_i)$$

Results

Running parameters ($\eta = 0.0003 \quad \epsilon = 50$) through the training loop yields the following convergence values:

Final Model Equation:

```
Step 8299 | a: -243.8114 | s: 7561.8465
Step 8300 | a: -243.8124 | s: 7561.8615
Step 8301 | a: -243.8133 | s: 7561.8765

---- final step reached ----
최종 수식: Traffic = (-243.8133 * Time) + (7561.8765)
```

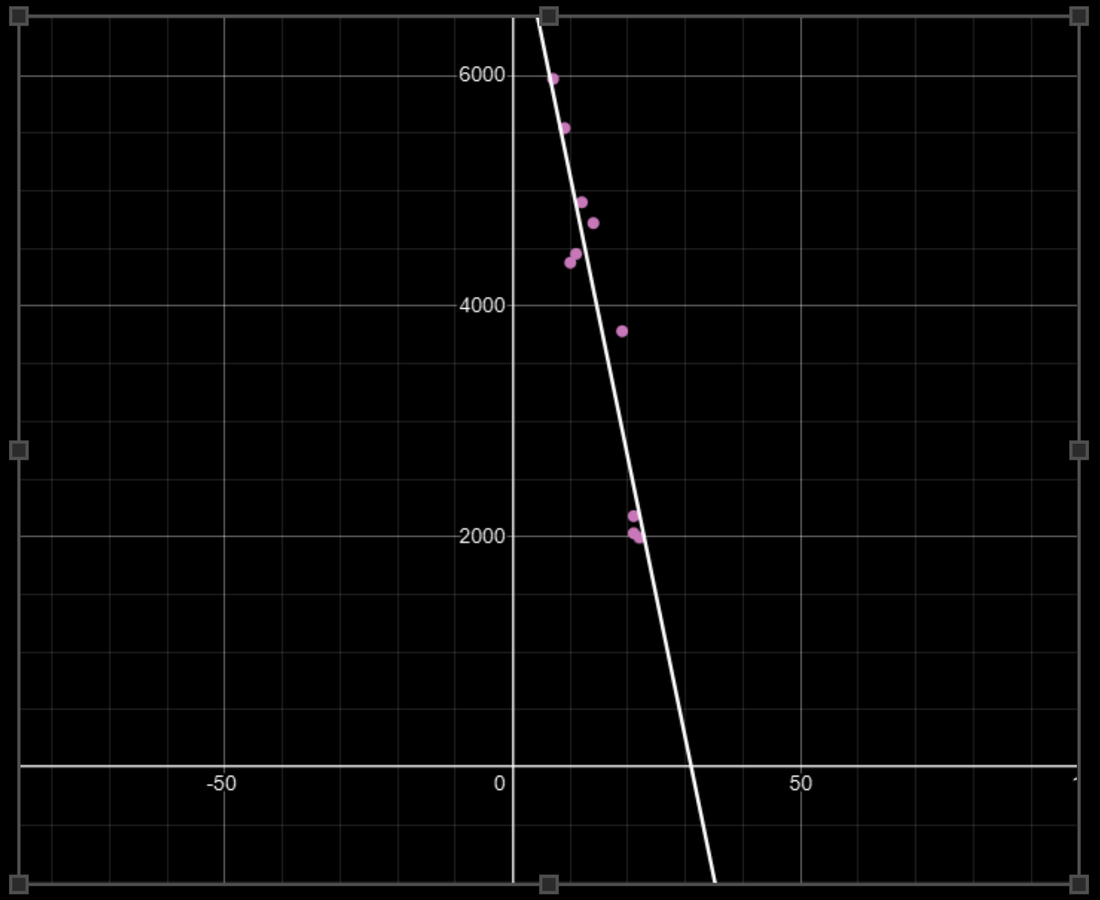
$$\text{Traffic Volume} = -243.8133 \cdot \text{Hour of Day} + 7561.8765$$

Interpretation: For every hour that passes later into the evening among these data points, the traffic decreases by approximately 284 vehicles per hour, starting from 07:00 AM
Domain Bounds Inquiry: the independent variable for time is strictly restricted to a minimum domain of 07:00 to 22:00.

Reason: Full 24-hour traffic volume is non-linear due to steep drop-off during early morning hours (01:00-06:00). Including these hours breaks the core assumption of **“linearity”** required for simple linear regession.

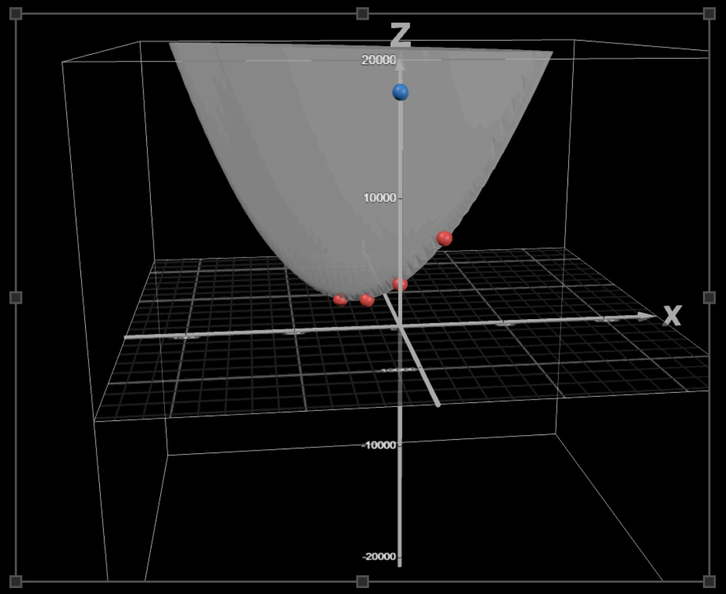
Graphs

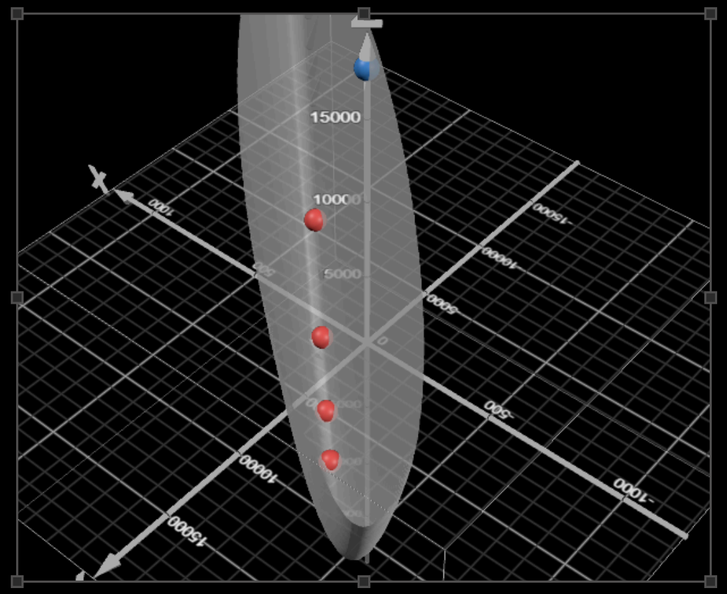
Scatterplots & Regression Line:



Graphs

Error Decrease Graph:





Technology Demonstrations

Open Desmos & JAVA VSC: ...

Code:

```
import java.io.File;
import java.io.IOException;
import java.util.ArrayList;
import java.util.Scanner;

public class GradientDescent
{
  public static void main(String[] args) throws IOException
  {
    ArrayList<DataPoint> dataList = new ArrayList<>();
    Scanner sc = new Scanner(new File("10points.txt"));
    while(sc.hasNextInt())
    {
      int time = sc.nextInt();
      int trafficVolume = sc.nextInt();
      dataList.add(new DataPoint(time, trafficVolume));
    }
    sc.close();

    runGradientDescentSteps(dataList);
  }

  public static void runGradientDescentSteps(ArrayList<DataPoint> dataList)
  {
    double eta = 0.0003;
    double epsilon = 50;
    double a = 1.0;
    double s = 0.0;

    double sumGradS;
    double sumGradA;
    int step = 1;

    System.out.println("---- Begin Gradient Descent training (do-while loop) ----");

    do
    {
      sumGradS = 0;
      sumGradA = 0;

      for (int i = 0; i < dataList.size(); i++)
      {
        int xi = dataList.get(i).time;
        int yi = dataList.get(i).trafficVolume;
        double pred = (a * xi) + s;

        sumGradS += 2 * (pred - yi);
        sumGradA += 2 * xi * (pred - yi);
      }

      s = s - (eta * sumGradS);
      a = a - (eta * sumGradA);

      System.out.printf("Step %3d | a: %8.4f | s: %8.4f\n", step++, a, s);


    }
    while (Math.abs(sumGradS) > epsilon || Math.abs(sumGradA) > epsilon);
    {
      System.out.println("\n---- final step reached ----");
      System.out.printf("최종 수식: Traffic = (%.4f * Time) + (%.4f)\n", a, s);
    }
  }

}
```

```
public class DataPoint
{
  public int time;
  public int trafficVolume;

  public DataPoint(int time, int trafficVolume)
  {
    this.time = time;
    this.trafficVolume = trafficVolume;
  }
}
```

threshold set to 50 because y value is multiple times greater than x value, so setting it to 0.1 would exhaust calculation process for slight additional precision

How Calculus Improves Predictions

Calculus Improvement:

- It systematically forces the Sum of Squared Residuals down to its lowest point
- It reduces the burden to calculate all the squared values by only requiring the calculation of linear derivative to approach to zero

How AI Learns

Old Memory (s_n, a_n) → current parameter guess

the Mistake (partial derivate values) → error slope measurement

Correction (- sign):

$$s_{n+1} = s_n - \left.\frac{\partial F}{\partial s}\right|_{s=s_n} \cdot \eta$$

step size (eta): learning speed control → if too high, it may cause infinity

the next value depends on the performance of the current value

$$a_{n+1} = a_n - \left.\frac{\partial F}{\partial a}\right|_{a=a_n} \cdot \eta$$

_(n+1) → acts as a new brain (upgraded, smarter model)

What derivatives do

Rate of Change: Measures how much the error function drops when parameters alter slightly

Near-Zero Target: Signals that the optimal minimum error is approximately reached when the gradient hits zero